

SUMMER CAMP • FINAL REPORT

GUI Agent 长程任务中的 记忆：从系统增益到信息归因

从显式任务账本的纵向改进，到事实级因果归因的分析转折

浙江大学夏令营课题最终汇报

汇报人：许奕 · 2026.08.19

WHAT I DID IN THIS PROJECT

本课题工作围绕一条主线：把“记忆是否有用”拆成可复现、可比、可解释的实验

01 • 先让结果可信

建立可审计基线

复现 19 个 AndroidWorld Hard 任务，补跑 3 个 seed；把基础设施错误与科学失败分开。

产出：57 / 57 个有效 episode + 完整轨迹



02 • 再探索设计空间

横向比较 + 纵向升级

横向移植进度、 workflow、图记忆与恢复机制；纵向从 A0、A1 迭代到最小账本 R2。

产出：A / R 系列正负对照 + 24 个 donor slot + 最强完整结果 6 / 19



03 • 最后解释为什么

从总分回到轨迹

R15 暴露“成功但新组件未读取”的矛盾，促使我们重新审计旧实验的信息断点。

产出：事实级信息机会链与可验证假设

最终回答 什么样的记忆能带来系统增益，以及我们如何知道增益真正来自它？

接下来按这个认识顺序展开：先看实验基础和设计，再看结果、矛盾与方法转折。

THE AUDITABLE BASELINE

A0 先建立可信起点：无显式任务账本，三 seed 为 4 / 2 / 1

同一 19 题集，完整成功数仍明显波动

20260806		4 / 19
20260807		2 / 19
20260808		1 / 19

7 / 57 完整成功 **57 / 57** 科学有效 episode

A0 输入与行为

当前截图 + 普通 Thought / Action 历史

- 没有跨步显式的 verified / pending 任务账本
- 动作执行、奖励和 step 轨迹均可审计
- 基础设施错误不进入科学结果

比较规则：后续单 seed 只支持同 seed 配对结论；单个高分不能代表稳定能力。

A0 不只是一个分数，而是后续每次改动的可比起点和证据校准器。

THE MAIN VERTICAL LINE

A0 → A1 → R2: 先让待办持续可见，再只保留真正有用的最小账本

A0 • 无显式任务状态

截图 + 普通历史

假设起点：长程任务中的未确认事项可能随历史增长而失去显著性。

4 / 19

开发 seed

A1 • Action Working Memory

最多六条 observed / verified / pending

理论：持续提醒已完成与未确认事项，避免最后一步被遗忘。

5 / 19

新增 Recipe; calls +83%, tokens +172%

R2 • Compact Verified / Pending

只留最新账本，删除 observed 与历史重复

理论：当前截图是权威；保留有效待办，同时减少陈旧状态和上下文负担。

6 / 19

保留五题 + Osm; token / time 下降

纵向设计逻辑：不是“记忆越多越好”，而是从无账本，到显式待办，再到选择性最小账本。

第一条主线已经完整：理论动机、具体改动、实际增益和成本变化一一对应。

HORIZONTAL EXPLORATION • WHAT WAS ACTUALLY TESTED

同时展开的横向探索说明：换一种记忆形式，并不会自动转化为任务成功

方向	实际改动	运行证据	结果	获得的认识
A2 可信进度	高频写入经过约束的进度摘要	高写入 / 高读取	0 / 19	更频繁、更可信的进度仍不是充分条件
A3 / A5 字段 / 图记忆	最小移植 MemGUI / HyMEM 式结构	不完整 / 未激活	0 / 1	说明最小外挂不足，不能外推为原论文无效
A4-v2 AWM donor bank	预注册每条路线至少 2 个成功 donor	24 / 24 slot 完成	3 / 7 路线达标	4 条路线在 4 / 4 机会耗尽后仍无法 induction
A6 持续轨迹回放	把更多历史持续放回上下文	机制过度活跃	0 / 19	更多记忆可能稳定重复动作与错误路线
A8-A12 停滞与恢复	检测循环、停滞并注入恢复提醒	有触发 / 有读取	0 个有益分叉	提醒“卡住了”不等于获得替代策略

A4-v2 最新封存：24 个有效 donor episode = 6 成功 + 18 科学失败；454 calls / 442 actions。因 4 条路线 coverage blocked，workflow induction 与七题评分按协议停止。

横向实验不只比较“记忆形式”：它进一步发现，纯记忆可能在信息、策略甚至可复用 donor 的前置条件上就被卡住。

MAIN SYSTEM RESULT

R2 用最小任务账本做到 6 / 19：系统增益与低复杂度同时成立

COMPACT VERIFIED + PENDING

6 / 19

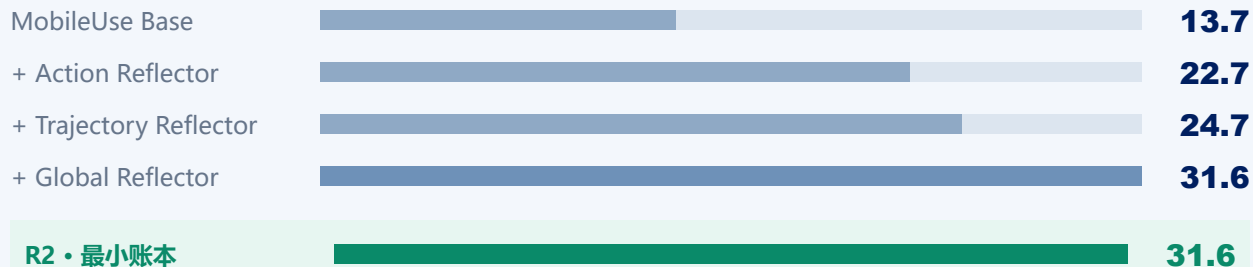
31.6% · AndroidWorld Hard

相对 A1: 1 胜 · 0 负 · 18 平

-22.5%
token-23.1%
time0
额外模型调用

NUMERICAL CONTEXT, NOT A LEADERBOARD

同一官方 Hard 难度划分的数值坐标



复杂度证据

R2: 单执行器 + 账本

MobileUse: Operator + Progressor + 三级 Reflector

完整分母 19 个 Hard 类混合记忆、多应用、复杂 UI、搜索、录入与计算需求

Sources: MobileUse, NeurIPS 2025, Table 4; official AndroidWorld task list. 不同 backbone、run、seed 与版本, 仅作数值和复杂度语境。

可守住的亮点：在混合能力的 19 个 Hard 任务上，轻量记忆干预达到 MobileUse 最强 Hard 消融的同一数值，同时不引入规划器、验证器或反思器调用。

WHY R2 STOPS AT 6 / 19

R2 解决了“待办是否可见”，但剩余 13 题多数卡在它没有增加的能力

7 / 13

耗尽步数，任务长期未解决

失败组平均 38.31 actions; MarkorMerge 与 OsmAndTrack 合计贡献 94 / 130 次 same-state refresh。

缺口：路线恢复、多对象 frontier、可执行替代策略

6 / 13

主动给出错误完成判断

3 个错误 terminate(success), 3 个错误 answer; 信息仍可能看见，但核验和 grounding 出错。

缺口：对象核验、答案检查、completion grounding

记忆能改善什么

让目标、待办和已核验事实持续可用。

记忆不能凭空生成什么

新路线、精确视觉定位和可靠完成验证。

由此产生的新问题

后续补丁没涨分，究竟是没生效，还是只修了局部？

到这里故事产生了真正的矛盾：我们知道 R2 的边界，也做了针对性补丁，但仅靠终分仍无法判断每次升级到底修到了哪里。

THE R15 TURNING POINT

R15 带来的不是更高总分，而是一个旧审计无法解释的成功

BROWSERMULTIPLY • THE TRIGGER

五个值被保留并算出 1120

 $1 \cdot x \cdot 8 \cdot x \cdot 10 \cdot x \cdot 7 \cdot x \cdot 2 = 1120$

R2 120 R13D 720 R14 2310 R15 1120

关键事实沿普通 Thought / Action 历史保留；R15 特有组件 read = 0。

FIXED SEVEN-TASK SCOPE CHECK

4 / 7

描述性 stitched panel

3 / 6

新补六题：Expense、Retro、Sports 成功

0

六题 opportunity / activation / read / use

0w · 3t ·
3r

相对历史 R2，且不是匹配对照

4 / 7 不能作为新方法分数，却能证明“任务成功”和“新组件获得信用”是两件事。

旧审计：组件写了什么、读了什么

→ R15 矛盾：组件没读，任务却成功

→ 新问题：关键事实实际经过了哪条路径？

因此 R15 的价值是改变分析起点：不再先问“哪个组件成功”，而是先问“任务所需事实是否一路保持可用”。

R15 让审计下沉到事实，A4-v2 又揭示：记忆在读取前就可能失去机会



这套分析从“组件有没有读”扩展为 P0 + L0-L6: 先审前置条件, 再追任务事实, 最后才评价动作与结果。

RE-READING THE OLD EXPERIMENTS

把新框架带回旧轨迹：同样是 0 分，实际修复点与剩余断点完全不同

升级链	仅看终分	轨迹中的真实变化	机会链断点	为什么仍未成功
R5 → R6 目标保持	0 → 0	原始目标重新进入上下文，goal 丢失被修复	L2-L4 已修	错误空间点击与弱恢复仍在 L5-L6
R7 → R9 循环恢复	0 → 0	R9 检测器实际触发 3 次	到达 L4	提醒未产生有效动作分叉，停在 L5
R10 → R11 对象核验	0 → 0	一次错误删除被纠正	局部 L5-L6	完整任务仍含其他对象与完成验证瓶颈
R11 → R12 去重压缩	0 → 0	去除 174 条重复记录，token -4.37%	成本侧改善	降低冗余不等于获得新的解题策略
过去只看到 10 次 0 / 1		现在能区分 未生效 / 到达无分叉 / 局部改善		下一次应做 修复最早断点

这页回答“新思维有什么用”：它把旧实验从十个相同失败，重构成不同层级的证据，并解释了为什么继续叠加通用字段不会自然超过 R2。

WHAT IS DIFFERENT, AND WHAT IS NOT YET PROVEN

与论文相比，我们增加的是“前置条件 + 事实级断点”的审计尺度

AWM • arXiv:2409.07429

从成功轨迹归纳 workflow

先获得成功 demonstrations，再 induction、检索与复用。

workflow / task / system

MEMGUI-BENCH • 2026

保持率与五类失败

用 IRR 和 memory-unit 指标衡量记忆。

任务 / challenge / memory unit

本项目 • FROZEN TRACE

P0 + 事实 × 阶段 × 轨迹

先审 donor / opportunity，再定位最早事实断点。

前置条件到结果的信用链

目前已经支持

描述性映射 + 回顾性断点区分

仍需决定性验证

盲法双人标注 + 预注册的瓶颈匹配干预

Sources: AWM, arXiv:2409.07429; MemGUI-Bench, arXiv:2602.06075; A4-v2 donor acquisition closure. 不声称该尺度普遍首次出现。

后续研究空间由此变得具体：记忆评测不仅要问“读到了吗”，还要测 donor coverage、信息机会与最早断点能否预测有效干预。

CONCLUSION

轻量系统增益，异质失败证据，可检验的分析转折

- 01 系统结果** R2 以最小 verified / pending 账本达到 6 / 19，在混合 Hard 任务上取得 31.6%，且零额外模型调用。
- 02 实验结论** 纵向说明记忆不是越多越好；A4-v2 进一步表明，可复用 donor coverage 本身也可能成为上游瓶颈。
- 03 分析转折** R15 与 A4-v2 把审计扩展为 P0 + L0-L6；回顾性断点诊断已有证据，前瞻预测仍待验证。

不是“记忆越多越好”，而是哪条事实、在哪一层、通过什么干预真正改变了动作。

Questions & Discussion